

SkyBridge Compass: Supplementary Materials

Zi'ang Li, Peng Liu
Independent Researcher, Tianjin, China
E-mail: 2403871950@qq.com
Independent Researcher, Qiqihar, China



SUPPLEMENTARY TABLES

This document provides extended data tables referenced in the main paper. All data is derived from the artifact repository CSV files.

Correspondence with Main Text:

- Table S12 (Latency): Referenced in main paper Section 7, supports Table 9 and Fig. 8
- Table S13 (RTT): Referenced in main paper Section 7, supports Table 9
- Table S14 (Message Sizes): Referenced in main paper Section 7, supports Table 9 and Fig. 9
- Table S24 (Loopback Wire Sizes): Supports Table 3 (baseline comparison)
- Tables S25–S26 (Repeatability): Supports Section 7. Tables report observed batch count B and (when $B \geq 2$) 95% confidence intervals across batches; set `SKYBRIDGE_BENCH_BATCHES=3–5` to reproduce multi-batch CIs.
- Table S15 (Traffic Padding): Supports Section 7 (traffic analysis mitigation quantization/overhead)
- Table S17 (Real-network micro-study): STUN path + 12 kB-class payload measurements across Wi-Fi / hotspot labels (generated from `Artifacts/realnet_*`). (Cross-NAT inbound-reachability depends on ISP/upstream equipment and may be infeasible without administrative control; see note below.)
- Table S18 (Dedicated EC2 direct-path validation): external Linux direct TCP run (no tunnel) with $n=50$ per payload for the pinned artifact date.
- Table S19 (Kernel-level emulation cross-check): `dummysnet/netem` comparison under shared profile IDs, supporting Section 7.7.
- Table S20 (iOS microbench metadata): per-device hardware/software/thermal metadata and run configuration for the schema v3 JSON artifacts supporting Table S21. For the frozen artifact date (2026-01-23), the iOS compatibility-matrix gate passes for the tested iPad/iPhone deployment set (no schema-version incompatibility observed).
- Table S22 (v1/v2 bridge comparison): liboqs PQC v1 baseline versus v2 FS latency/RTT and payload-byte delta, supporting main-paper Section 7.
- Tables S31–S33 (Platform/runtime matrix): Apple provider availability plus Android/Ubuntu version-tier suite policy mapping used by negotiation.
- Table S32 (Cross-platform contract consistency): static source-based checker output across iOS/macOS, Android, and Ubuntu for suite parity, encoding, fallback guardrails, and trust path.
- Table S23 (Cross-platform interop matrix): measured external Linux direct/tunnel rows plus static Android/Ubuntu contract-consistency pair rows under a shared artifact date.
- Table S38 (Boundary-stress cases and operator runbook): explicit in-model vs out-of-model classification for pairing social engineering, endpoint compromise classes, and audit-log abuse, with expected event signals and required operator actions.
- Table S39 (Formal-to-code traceability): mapping of theorem-facing lemmas/events to implementation-level enforcement points and artifacts.
- Tables S27–S40 (Protocol/Audit Appendices): wire-format validation rules, suite identifiers, PQC coverage and platform support matrices, key-size references, security event taxonomy, and an implementation-to-file mapping. These materials support auditability and reproducibility claims in main paper Sections 4, 5, and 6.
- Tables moved from the main paper to satisfy page budget: Table S1, Table S2, Table S3, Table S4, Table S5, Table S6, Table S9, Table S10, Table S11, Table S21, Table S7, Table S8.

TABLE S1
Reviewer-facing novelty boundary: prior-art defaults vs. claimed system-level delta.

Capability axis	Prior-art default (protocol/framework level)	Delta claimed in this work	Evidence anchor
Negotiation and transcript binding	Mature protocol support exists (e.g., TLS/QUIC/Noise transcript integrity)	Reused foundation; not claimed as novel by itself	Table 3
Policy non-bypass at entry points	Usually enforced in caller/application glue	Policy-bound session API + compile/runtime guards to block bypass paths	Section 6; Supplementary Regression Testing
Fallback eligibility semantics	Downgrade prevention exists, but fallback causes and scope are often implementation-defined	Local-only, pre-send eligibility boundary; timeout/network-derived causes are fallback-ineligible	Table 5; Fig. 5
Audit semantics for downgrade vs failure	Alert/event schemas are commonly externalized	Downgrade-labeled events only on accepted fallback; non-fallback failures remain typed but not downgrade-labeled	Table 2; Fig. 7
Operator strictness realizations	Single strictness mode or ad hoc compatibility behavior	Two explicit strict modes (Compliance / Bootstrap-Assisted) with mandatory PQC rekey gate before business traffic	Fig. 4; Section 6

TABLE S2
PQC migration design space (trigger origin vs evidence strength).

Approach class	Fallback trigger origin (typical)	Evidence strength (typical)	Migration-security implication
TLS/QUIC deployment defaults	Handshake/protocol errors are explicit, but migration fallback causes are largely implementation-defined in surrounding system glue	Transcript integrity is strong; system-level fallback evidence schema is not standardized	Protocol-correct deployments can still be migration-unsafe operationally if timeout/network-derived fallback is admitted in implementation policy
Noise-style fixed patterns	Usually fixed-pattern negotiation; migration/fallback behavior delegated to application framework	No built-in migration audit schema; evidence externalized to application logs	Strong AKE pattern semantics, but migration policy enforceability and replayable downgrade evidence are out of scope
SkyBridge migration contract (this work)	Local-only whitelist for fallback; timeout/network-derived causes are fallback-ineligible by definition	Structured, replayable downgrade evidence (D3 fields) plus deterministic policy decision semantics	Enforceable migration gate with operator-auditable downgrade decisions under active suppression model

TABLE S3
Claim-to-evidence strength map (main-paper claims only).

Claim axis	Evidence method	Scale / sample scope	Artifact / section anchor
Negotiation integrity and transcript binding	Formal proof obligations + protocol/property tests	Theorem/Proposition set + reproducible seeded tests	Section 5; formal/skybridge_v2.spthy; Section 7
Policy-gated downgrade correctness	Decision-matrix validation + forced-error policy bench + migration-threat harness contrast	N=1000 per policy/scenario family; Wilson CI reported	Fig. 4; Fig. 5; Table S9; Table 6; Table S4; policy-downgrade + migration-threat CSV artifacts
Deterministic failure semantics and event typing	Fault injection + spec-to-event trace mapping	Fault injection n=1000 per scenario; MR=1.00/SR=0.00 in reported classes	Table S6; Table 8; fault-injection CSV artifact
Performance and overhead characterization	Distributional microbench + system-level amortization	Classic/liboqs N=1000, CryptoKit N=5000; multi-batch CI support	Table 9; Fig. 10; Section 7.6
External-validity guardrail (deployment instance)	Controlled simulation + kernel cross-check + fleet-level migration dynamics + real-network micro-study	Controlled N=10,000/condition; dedicated direct-path external run n=50/payload	Table S8; Table S5; Section 7.7; Supplementary Tables S19, S18, S23; migration-fleet/interoperability CSV artifacts

TABLE S4
Provider/back-end migration-threat matrix (N=1000 per row). Data source: Artifacts/
migration_threat_provider_matrix_2026-01-23.csv produced by MigrationThreatProviderMatrixBenchTests.

Provider	Policy	Scenario	FDR	SDR	ECR	PCR	Interpretation
liboqs	default	timeout attack	0.00	0.00	1.00	1.00	Timeout suppression does not create a classic edge
liboqs	strictPQC	timeout attack	0.00	0.00	1.00	1.00	Strict mode remains fail-closed under timeout suppression
liboqs	default	local unavailability	1.00	0.00	1.00	1.00	Local whitelist fallback is explicit and fully evidenced
classic	default	PQC provider unavailable	1.00	0.00	1.00	1.00	No-PQC backend path still emits replayable downgrade evidence

Main-paper Tables Moved to Supplementary (Page Budget)

Note: Cross-NAT direct inbound tests (no overlay/relay) are not universally feasible. In particular, double-NAT topologies (ISP router/modem upstream) and IPv6 inbound filtering can prevent any inbound reachability even when devices have global IPv6 addresses. In such environments, all client samples time out with empty connect-time fields, and we omit those rows from the main table to avoid conflating “connectivity unavailable” with protocol performance.

Wire Size Summary:

- Classic: $293 + 318 + 2 \times 38 = 687$ B
- PQC (liboqs): $6507 + 5419 + 2 \times 38 = 12,002$ B
- PQC (CryptoKit): $6514 + 5419 + 2 \times 38 = 12,009$ B

TABLE S5
Fleet-level migration dynamics under heterogeneous PQC capability distributions (N=10,000 sessions per row).

PQC-capable ratio	Cooldown	Success rate	Downgrade events / 1k	Mean connect (ms)	Decision-share breakdown (PQC / classic / rate-limited / timeout)
20%	on	0.3830	195.60	0.009	0.1874 / 0.1956 / 0.6066 / 0.0104
20%	off	0.9899	799.80	0.010	0.1901 / 0.7998 / 0.0000 / 0.0101
50%	on	0.6589	185.70	0.006	0.4732 / 0.1857 / 0.3170 / 0.0241
50%	off	0.9761	500.80	0.007	0.4753 / 0.5008 / 0.0000 / 0.0239
80%	on	0.8867	124.80	0.004	0.7619 / 0.1248 / 0.0733 / 0.0400
80%	off	0.9595	204.80	0.005	0.7547 / 0.2048 / 0.0000 / 0.0405

TABLE S6
Failure-Mode Robustness and Observability. Runs = number of iterations per scenario and policy (n=1000). Table reports default policy; strictPQC (Compliance Mode) counts are recorded separately in `fault_injection_2026-01-23.csv`. All metrics are binary (1 = pass) except event counts. Data from `Artifacts/fault_injection_2026-01-23.csv` produced by `HandshakeFaultInjectionBenchTests`; semantic checks from `HandshakeDriverTests`.

Failure Mode	Runs	No Unexp. Error	No Double Resume	Zero Verified	$E_{\text{handshakeFailed}}$	$E_{\text{cryptoDowngrade}}$
Out-of-order	1000	1	1	1	0	0
Duplicate	1000	1	1	1	0	0
Drop	1000	1	1	1	1000	0
Delay within timeout	1000	1	1	1	0	0
Delay exceed timeout	1000	1	1	1	1000	0
Corrupt header	1000	1	1	1	1000	0
Corrupt payload	1000	1	1	1	1000	0
Wrong signature	1000	1	1	1	1000	0
Concurrent cancel	1000	1	1	1	1000	0
Concurrent timeout	1000	1	1	1	1000	0

TABLE S7
Handshake vs data-plane AEAD selection (v1).

Layer	AEAD	Negotiation source
Handshake payloads	AES-256-GCM	Suite tuple
Data-plane traffic	AES-256-GCM	Fixed (v1)

TABLE S8
Controlled in-process network simulation results (N=10,000 per condition, max retries=2). Completion and latency ranges span classic/liboqs/CryptoKit suites.

Condition	Completion	p95 latency	handshake
Failed (loss/RTT/jitter/reorder)	(min)	(ms, range)	/10k (max)
mild (1%, 50 ms, ± 20 ms)	1.0000	251–253	0
moderate (3%, 100 ms, ± 50 ms)	1.0000	529–532	0
severe (5%, 200 ms, ± 100 ms)	0.9989	1031–1033	11
reorder (10% reorder, 50 ms, ± 20 ms)	1.0000	368–370	0

TABLE S9
Ablation-by-property summary (artifact-backed).

Contract mechanism	Ablated variant	Observed breakage	Evidence anchor
Local-only fallback gate + timeout blacklist (D2)	Timeout/network-derived fallback allowed	Under timeout attack, FDR/SDR/ECR/PCR shifts from 0.00/0.00/1.00/1.00 to 1.00/1.00/0.00/0.00 (attacker-forced classic with silent downgrade)	Table 6
Structured downgrade evidence + replay fields (D3)	Classic fallback without downgrade event schema	Fallback becomes non-auditable and non-replayable (SDR=1.00, PCR=0.00 on the baseline row)	Table 6
Per-peer fallback cooldown gate	Cooldown disabled in heterogeneous fleet	Downgrade exposure increases from 195.6 to 799.8/1k (20% PQC-capable), 185.7 to 500.8/1k (50%), and 124.8 to 204.8/1k (80%)	Supplementary Table S5

TABLE S10
Suite negotiation scenarios under the two-attempt strategy (homogeneous per attempt).

Initiator attempt(s)	Responder Capability	Outcome	Notes
Attempt 1: PQC-only	PQC available	PQC established	Best available selected
Attempt 1: PQC-only; Attempt 2: classic-only	Classic only (no PQC material)	Classic established	Graceful fallback (evented)
PQC-only (strictPQC, bootstrap-assisted)	Missing/stale paired KEM key	PQC established after rekey	Classic channel is control-only before rekey
PQC-only (strictPQC)	Classic only	Handshake failed	Policy enforced (no fallback)
Classic-only	PQC available	Classic established	Initiator policy respected

TABLE S11
System-level impact and amortization (p50/p95, ms). $T_{connect}$ is measured from connect start to handshake established (including Finished key confirmation and event emission). T_{total} is measured from connect start to completion of an encrypted bulk transfer. Network conditions emulate deterministic RTT+jitter on a reliable stream; large data-plane payloads are bandwidth-limited (MiB/s) to avoid per-chunk RTT artifacts. Runs: $N_{connect}=1000$; N_{total} (RTT 50±20 ms) = 1MiB:N=1000, 10MiB:N=500, 100MiB:N=100. Data from Artifacts/system_impact_2026-01-23.csv.

Suite	$T_{connect}$ (p50/p95)			T_{total} (p50/p95), RTT 50±20 ms		
	ideal	RTT 50±20 ms	RTT 100±50 ms	1 MiB	10 MiB	100 MiB
Classic	11.9/12.2	191.9/198.6	407.8/414.6	378.3/395.6	634.8/657.3	3179/3206
liboqs PQC	14.0/15.1	195.1/202.1	415.4/430.9	372/410	638.4/672.6	3192/3583
CryptoKit PQC	16.3/18.4	196.2/205.8	412.1/426.8	371/434	581.0/593.2	3208/3285

TABLE S12
Supplementary Table S12: Full Handshake Latency Statistics.

Configuration	N	mean (ms)	std (ms)	p50 (ms)	p95 (ms)	p99 (ms)
Classic	1000	2.400	0.093	2.366	2.617	2.759
liboqs PQC	1000	2.954	0.387	2.875	3.689	4.155
CryptoKit PQC	5000	5.329	0.473	5.242	6.216	6.802

TABLE S13
Supplementary Table S13: Full RTT Statistics.

Configuration	N	mean (ms)	p50 (ms)	p95 (ms)	p99 (ms)
Classic	1000	0.529	0.517	0.579	0.635
liboqs PQC	1000	0.959	0.886	1.453	1.823
CryptoKit PQC	5000	2.147	2.061	2.776	3.320

TABLE S14
Supplementary Table S14: Message Size Breakdown by Field.

Message	Total (B)	Signature (B)	KeyShare (B)	Identity (B)	Overhead (B)
MessageA.Classic	293	64	32	36	161
MessageB.Classic	318	64	32	36	186
MessageA.PQC-liboqs	6507	3309	1088	1956	154
MessageB.PQC-liboqs	5419	3309	0	1956	154
MessageA.PQC-liboqs-v2fs	6550	3309	1088	1956	197
MessageB.PQC-liboqs-v2fs	5451	3309	32	1956	154
MessageA.PQC-CryptoKit	6514	3309	1088	1956	161
MessageB.PQC-CryptoKit	5419	3309	0	1956	154
MessageA.XWing	6641	3309	1216	1956	160
MessageB.XWing	5419	3309	0	1956	154
MessageA.PQC	6507	3309	1088	1956	154
MessageB.PQC	5419	3309	0	1956	154
Finished	38	0	0	0	38

- X-Wing (2026-01-23 snapshot): $6641 + 5419 + 2 \times 38 = 12,136$ B
- Real-network micro-study payload labels use the payload-only sizes above; the transport harness adds a fixed 4-byte length

TABLE S15

Supplementary Table S15: SBP2 traffic padding quantization summary. “raw”/“padded” are aggregate bytes across events; overhead is relative to raw. Top bucket reports the most frequent bucket size (share).

Label	wraps	unwraps	raw (B)	padded (B)	overhead (%)	top bucket
rx	0	112000	736918806	1451759104	97%	256B (37%)
CP/fileTransferRequest	7991	0	2916715	4091392	40%	512B (100%)
CP/heartbeat	8069	0	451864	2065664	357%	256B (100%)
CP/systemCommand	7940	0	1675340	2032640	21%	256B (100%)
DP/32B	4792	0	325856	1226752	276%	256B (100%)
DP/300B	4768	0	1602048	2441216	52%	512B (100%)
DP/900B	4839	0	4529304	4955136	9%	1024B (100%)
DP/1400B	4764	0	6841104	9756672	43%	2048B (100%)
DP/4KiB	4837	0	19986484	39624704	98%	8192B (100%)
DP/16KiB	3956	0	64957520	129630208	100%	32768B (100%)
DP/64KiB	4044	0	265173168	530055168	100%	131072B (100%)

TABLE S16

Supplementary Table S16: SBP2 bucket-cap sensitivity study. We vary the maximum bucket size (cap) and report padding overhead, cap coverage (fraction of frames whose framed payload exceeds the cap), and a privacy proxy (bucket entropy) for representative handshake, control, and data-plane workloads.

Label	64 KiB cap				128 KiB cap				256 KiB cap			
	all (%)	padded (%)	>cap (%)	entropy (b)	all (%)	padded (%)	>cap (%)	entropy (b)	all (%)	padded (%)	>cap (%)	entropy (b)
HS/MessageA	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
HS/MessageB	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
HS/Finished	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
CP/heartbeat	357%	357%	0%	0.00	357%	357%	0%	0.00	357%	357%	0%	0.00
CP/systemCommand	21%	21%	0%	0.00	21%	21%	0%	0.00	21%	21%	0%	0.00
CP/fileTransferRequest	40%	40%	0%	0.00	40%	40%	0%	0.00	40%	40%	0%	0.00
DP/32B	276%	276%	0%	0.00	276%	276%	0%	0.00	276%	276%	0%	0.00
DP/300B	52%	52%	0%	0.00	52%	52%	0%	0.00	52%	52%	0%	0.00
DP/900B	9%	9%	0%	0.00	9%	9%	0%	0.00	9%	9%	0%	0.00
DP/1400B	43%	43%	0%	0.00	43%	43%	0%	0.00	43%	43%	0%	0.00
DP/4KiB	98%	98%	0%	0.00	98%	98%	0%	0.00	98%	98%	0%	0.00
DP/16KiB	100%	100%	0%	0.00	100%	100%	0%	0.00	100%	100%	0%	0.00
DP/rdpMix	1%	7%	79%	2.57	15%	32%	36%	1.78	36%	36%	0%	1.47
DP/fileMix	<0.1%	n/a	100%	2.58	12%	56%	64%	2.22	49%	49%	0%	0.95

Notes: *all* is overhead across all wrapped frames (including passthrough); *padded* is overhead restricted to frames that entered bucket padding. *n/a* denotes “not applicable” (handshake messages are padded by SBP1 rather than SBP2) or “no padded frames observed” for that cap/label. – denotes “not observed” in the sampled protocol trace. The >cap rate is computed as the fraction of frames where (payload bytes + SBP2 header) exceeds the configured cap; this can drive *all* toward near-zero while preserving non-zero >cap.

TABLE S17

Supplementary Table S17: Real-network micro-study (STUN path + TCP payload). Each row is one network condition label. STUN metrics capture path RTT/loss and a conservative NAT classification. E2E metrics report success rate and p50 completion time for two payload sizes (classic 687 B vs PQC 12,002 B), along with the delta (PQC minus classic). A fixed 4-byte transport length prefix is excluded from the payload-byte labels. Failure taxonomy is summarized for the PQC size as timeout rate and other failure rate. Labels with zero successful E2E samples on both payload sizes are omitted because timing percentiles are undefined.

Label	IFace	Exp	Con	NAT	STUN loss	STUN RTT	p50/p95 (ms)	ok _c	p50 _c (ms)	ok _p	p50 _p (ms)	Δp50 (ms)	PQC fail (to/other)
ec2_ssh_tunnel	wifi	0	0	unknown	0.1200		62.301/64.751	1.0000	204.607	1.0000	287.435	82.828	0.0000/0.0000
hotspot_ssh_tunnel	wifi	1	1	unknown	0.3200		223.066/426.633	1.0000	240.546	1.0000	280.219	39.673	0.0000/0.0000
wifi_direct	wifi	0	0	unknown	0.1800		166.881/169.392	1.0000	139.747	1.0000	165.242	25.495	0.0000/0.0000

TABLE S18

Dedicated EC2 direct-path validation (external Linux, no tunnel, ARTIFACT_DATE=2026-01-23). Endpoint: 54.92.79.99:44444; harness: Scripts/run_real_network_e2e_linux.py; n=50 per payload (PQC payload label 12,002 B is the liboqs payload-only handshake; CryptoKit is 12,009 B in this snapshot).

Payload (B)	Success rate	p50 total (ms)	p95 total (ms)	p99 total (ms)
687	1.0000	130.601	139.426	140.970
12,002	1.0000	132.925	140.858	148.236

TABLE S19

Supplementary Table S19: Kernel-level network emulation cross-check (shared profiles from `Scripts/netem_profiles.json`). Common profiles (mild/moderate/severe) are reported for both dummynet and netem when available; reorder is netem-only and marked *n/a* for dummynet by design. On some macOS setups, dummynet with localhost endpoint can under-shape absolute delay, so cross-tool interpretation emphasizes completion robustness over millisecond parity unless a non-localhost endpoint is used. Data from `Artifacts/network_emulation_kernel_2026-01-23.csv`.

Profile	Suite	dummynet comp.	dummynet p95	dummynet p99	netem comp.	netem p95	netem p99
mild	Classic	1.0000	273.8	289.7	1.0000	253.4	1221.8
mild	liboqs PQC	1.0000	316.0	366.1	1.0000	240.5	1214.6
mild	CryptoKit PQC	1.0000	347.8	470.6	1.0000	249.0	1229.8
moderate	Classic	1.0000	277.6	293.8	1.0000	1431.0	1513.9
moderate	liboqs PQC	1.0000	314.5	408.4	1.0000	1371.4	1573.1
moderate	CryptoKit PQC	1.0000	300.9	310.6	1.0000	1332.6	1475.5
severe	Classic	1.0000	277.4	305.6	1.0000	1889.0	3284.1
severe	liboqs PQC	1.0000	312.2	326.2	1.0000	1933.5	2126.9
severe	CryptoKit PQC	1.0000	308.6	315.0	1.0000	1897.2	3908.0
reorder	Classic	<i>n/a</i>	<i>n/a</i>	<i>n/a</i>	1.0000	233.8	253.3
reorder	liboqs PQC	<i>n/a</i>	<i>n/a</i>	<i>n/a</i>	1.0000	232.8	242.0
reorder	CryptoKit PQC	<i>n/a</i>	<i>n/a</i>	<i>n/a</i>	1.0000	241.3	253.3

TABLE S20

Supplementary Table S20: iOS micro-benchmark device metadata (`Artifacts/ios_microbench_2026-01-23.csv`).

Device label	Model	System	Build	Chip/machine	Thermal	Config
ipad16_3	iPad	26.3	Version 26.3 (Build 23D127)	iPad16,3	nominal	warmup=10, N=1000, B=3
iphone17_1	iPhone	26.2.1	Version 26.2.1 (Build 23C71)	iPhone17,1	fair	warmup=10, N=1000, B=3

TABLE S21

On-device micro-benchmark on iOS 26+ (two devices, schema v3 artifacts). Each device runs warmup=10, N=1000, batches=3. Cells report mean/p99 in ms from pooled measured iterations. Data from `Artifacts/ios_microbench_2026-01-23.csv` (generated via `Scripts/aggregate_ios_microbench.py`).

Configuration	ipad16_3				iphone17_1			
	KEM encap	Sign	Verify	KEM-DEM	KEM encap	Sign	Verify	KEM-DEM
Classic (X25519 + Ed25519)	0.182 / 7.314	0.105 / 0.072	0.067 / 0.049	0.388 / 7.588	0.190 / 7.479	0.111 / 0.086	0.066 / 0.046	0.426 / 8.052
CryptoKit PQC (ML-KEM-768 + ML-DSA-65)	0.131 / 0.097	1.771 / 8.396	0.103 / 0.071	0.333 / 7.519	0.143 / 0.272	3.135 / 10.124	0.125 / 0.091	0.387 / 8.232
CryptoKit Hybrid (X-Wing + ML-DSA-65)	0.215 / 7.375	1.671 / 8.454	0.106 / 0.072	0.424 / 7.558	0.255 / 8.108	2.711 / 9.960	0.127 / 0.096	0.520 / 8.591

TABLE S22

Supplementary Table S22: liboqs PQC v1/v2 bridge comparison (N=1000, warmup=10). v1 uses ML-KEM-768; v2 uses ML-KEM-768-FS with additional ephemeral contribution composition. Observed v2-v1 deltas: mean latency +0.77 ms, p95 latency +0.74 ms, payload bytes +75 B.

Configuration	mean latency (ms)	p95 latency (ms)	p50 RTT (ms)	p95 RTT (ms)	payload handshake (B)
liboqs v1	2.95	3.69	0.89	1.45	12,002
liboqs v2 FS	3.72	4.43	1.23	1.82	12,077

TABLE S23

Cross-platform interop matrix (measured external Linux paths + static contract-consistency pairs).

Initiator	Responder	Evidence	Suite mode	Path label	<i>n</i>	Success	p50 (ms)	p95 (ms)
iOS/macOS client	Ubuntu EC2	measured	classic	direct-ec2-54	50	1.0000	210.391	262.526
iOS/macOS client	Ubuntu EC2	measured	pqc-v1	direct-ec2-54	50	1.0000	225.216	334.648
iOS/macOS client	Ubuntu EC2	measured	classic	ssh-tunnel	50	1.0000	192.826	257.036
iOS/macOS client	Ubuntu EC2	measured	pqc-v1	ssh-tunnel	50	1.0000	267.350	338.673
iOS/macOS	Android	static_contract	catalog+policy-contract	static-contract	1	1.0000	-	-
iOS/macOS	Ubuntu	static_contract	catalog+policy-contract	static-contract	1	1.0000	-	-
Android	Ubuntu	static_contract	catalog+policy-contract	static-contract	1	1.0000	-	-

Notes: Measured rows use `Artifacts/realnet_e2e_summary_2026-01-23_*.csv`; static-contract rows derive from `Scripts/check_cross_platform_interop.py` output.

prefix that is excluded from these labels.

Table S24: Loopback Wire-Size Baselines

Values are cert-dependent (localhost certificate). Noise-XX wire size is pattern-dependent and reflects the XX pattern used in this baseline. SkyBridge loopback wire sizes include framing and transport overhead (and may include SBP1 handshake padding); payload-

TABLE S24
Loopback wire-size baselines from pcap capture (N=1000). Wire bytes count full packet lengths observed on the loopback interface; TLS/QUIC/DTLS values are cert-dependent.

Protocol	Wire p50 (B)	Wire p95 (B)
TLS 1.3	8,824	8,948
QUIC	7,830	14,427
WebRTC DTLS	3,090	3,162
Noise XX	2,560	2,672
SkyBridge (classic)	4,874	5,028
SkyBridge (liboqs)	37,779	44,609
SkyBridge (CryptoKit)	19,009	19,257

TABLE S25
Supplementary Table S25: Repeatability across independent benchmark batches (latency). Table reports observed batch count B . Cells report mean and (when $B \geq 2$) $\pm$ 95% CI across batches; configuration-specific N after 10 warmup runs (Classic=1000, liboqs PQC=1000, CryptoKit PQC=5000).

Configuration	B	N/batch	mean (ms)	p50 (ms)	p95 (ms)
Classic	5	1000	2.418 $\pm$ 0.045	2.368 $\pm$ 0.007	2.616 $\pm$ 0.007
liboqs PQC	5	1000	2.960 $\pm$ 0.013	2.873 $\pm$ 0.008	3.694 $\pm$ 0.073
CryptoKit PQC	5	5000	5.643 $\pm$ 1.154	5.548 $\pm$ 1.155	6.551 $\pm$ 1.174

TABLE S26
Supplementary Table S26: Repeatability across independent benchmark batches (RTT). Table reports observed batch count B . Cells report mean and (when $B \geq 2$) $\pm$ 95% CI across batches; configuration-specific N after 10 warmup runs (Classic=1000, liboqs PQC=1000, CryptoKit PQC=5000).

Configuration	B	N/batch	mean (ms)	p50 (ms)	p95 (ms)
Classic	5	1000	0.531 $\pm$ 0.012	0.518 $\pm$ 0.006	0.583 $\pm$ 0.038
liboqs PQC	5	1000	0.968 $\pm$ 0.031	0.897 $\pm$ 0.025	1.483 $\pm$ 0.098
CryptoKit PQC	5	5000	2.577 $\pm$ 1.135	2.483 $\pm$ 1.126	3.211 $\pm$ 1.144

only sizes are reported in Table S14.

Table S25: Repeatability Across Batches (Latency)

Table S26: Repeatability Across Batches (RTT)

Interpretation note: CryptoKit PQC RTT exhibits higher *between-batch* variance than Classic/liboqs in our environment (Apple Silicon, macOS 26.x), so its across-batch 95% CI can be substantially wider. Per-batch iteration counts are configuration-dependent in this artifact snapshot (Classic/liboqs N=1000, CryptoKit N=5000). This reflects run-to-run scheduler/load sensitivity rather than an arithmetic error; raw batch values are directly recorded in `Artifacts/handshake_rtt_2026-01-23.csv`.

Data Sources

All data is generated from the artifact CSV files:

- `Artifacts/handshake_bench_2026-01-23.csv` (Tables S12, S25)
- `Artifacts/handshake_rtt_2026-01-23.csv` (Tables S13, S26)
- `Artifacts/message_sizes_2026-01-23.csv` (Table S14)
- `Artifacts/traffic_padding_2026-01-23.csv` (Table S15)
- `Artifacts/traffic_padding_sensitivity_2026-01-23.csv` (Table S16)
- `Artifacts/network_emulation_kernel_2026-01-23.csv` (Table S19)
- `Artifacts/ios_microbench_2026-01-23.csv` (Table S21, Table S20)
- `Artifacts/realnet_microstudy_2026-01-23.csv` (Table S17)
- `Artifacts/interop_consistency_2026-01-23.json` and `Artifacts/interop_cross_platform_2026-01-23.csv` (Table S32, Table S23)
- `Artifacts/baseline_summary_*.csv` (Table S24)

Regenerate the benchmark CSVs with `Scripts/run_paper_eval.sh` from the artifact repository. Then build all paper/supplementary tables with `Scripts/make_tables.py`, which uses the main-paper pinned artifact date (or explicit `ARTIFACT_DATE`) to avoid mixing datasets across runs. System-level impact artifacts (`system_impact_...`) are produced by a dedicated suite, but follow the same artifact-date suffix convention used in the main paper.

Evidence Layering for This Revision

Main-paper claims and all primary figures/tables are anchored to the 2026-01-23 dataset family listed above. Any expanded checks introduced during revision are reported as **additional validation** in Supplementary sections and are used to strengthen robustness discussion; they do not replace or redefine the main-paper primary statistics.

Artifact release:

- URL: <https://github.com/billlza/Skybridge-Compass>
- Git ref: 251a162eb97d
- Commit: 251a162eb97d
- Source archive: 251a162eb97d.zip SHA256=d851c1ea3a256fde163a2f0f278bbe514b77d4694369c8352c99dbbfceaf6381
- Source archive: 251a162eb97d.tar.gz SHA256=e3630ddcad58f1f5b487aad3e72443f32e1eeea5fa9113603db84c61a9421562

SUPPLEMENTARY METHODS AND DETAILS

Wire Format and Validation Rules

Key Share Semantics: The `keyShares[]`.`shareBytes` field has suite-dependent interpretation:

- **DH suites (X25519):** `shareBytes` = ephemeral public key (32 bytes)
- **KEM suites (ML-KEM-768):** `shareBytes` = encapsulated key / ciphertext (enc, 1088 bytes)

This distinction matters: for DH, the Responder uses the Initiator’s public key to compute a shared secret; for KEM, the Initiator encapsulates to the Responder’s long-term KEM public key (obtained during pairing), and `shareBytes` carries the resulting enc. The Responder decapsulates using their private key.

Forward Secrecy Note: This is a static KEM exchange to a long-term KEM public key, and thus does *not* provide traditional PFS semantics. If the long-term KEM private key is compromised later, a passive attacker who recorded ciphertexts may be able to recover past session keys.

Nonce Freshness: Each party contributes a 32-byte nonce (`clientNonce` in A, `serverNonce` in B). Both are bound into the KDF info parameter, ensuring symmetric freshness and enabling a unique session identifier:

```
handshakeId = SHA256(replayTag ||
  initiatorNonce || responderNonce ||
  suiteWireIdLE)
```

To prevent short-window replay attacks, implementations **SHOULD** cache recent `handshakeId` values and reject duplicates within a configurable window (default: 5 minutes). Alternatively, they may cache the (`initiatorNonce`, `responderNonce`) pair.

Key Share Binding:

- The `keyShares[]` array contains at most two entries to bound message size. Under the two-attempt strategy, each `MessageA` is homogeneous (PQC/hybrid-only or classic-only), so the two entries (if present) belong to the same suite group for that attempt.
- Each entry is a (`suiteId`, `shareBytes`) tuple.
- The Responder **MUST** select a suite for which the Initiator provided a key share; otherwise reject with `missingKeyShare`.
- This binds negotiation to actual cryptographic material, preventing the “TLS key_share mismatch” class of bugs.

Explicit Key Confirmation (Finished Frames):

- The Responder sends a short `Finished_R2I` frame authenticated under the newly derived session keys. The Initiator verifies it and replies with `Finished_I2R`.
- A session is established only after both Finished frames are verified, reducing responder-side half-open state under failures.
- Finished frames are fixed-size authenticated messages (38 bytes each: 4-byte magic, 1-byte version, 1-byte direction, 32-byte HMAC), adding negligible wire overhead compared to PQC payloads.
- Finished MACs are computed over the handshake transcript using per-direction keys: `finishedMAC = HMAC-SHA256(finishedKey, transcriptHash)`
`finishedKey_R2I = HKDF(sessionKey, info="SkyBridge-FINISHED|R2I|")`
`finishedKey_I2R = HKDF(sessionKey, info="SkyBridge-FINISHED|I2R|")`

Anti-Downgrade Invariant:

- The Initiator **MUST** verify that `selectedSuite` is a member of `supportedSuites[]` it originally sent.
- It **MUST** also confirm that `keyShares[]` contains an entry for `selectedSuite`.
- Since `sigB` commits to `MessageA` via `transcriptA`, it binds the initiator’s proposal.
- Any tampering with the initiator’s offered suites or key shares will cause `sigB` verification to fail.

Canonical Encoding Rules (V1 Wire Format):

- `supportedSuites[]`: preference order, signed as-is (first = most preferred)

TABLE S27
Message field validation rules.

Field	Validation	Failure Action
version	Must equal protocol version (1)	Reject with versionMismatch
supportedSuites	Must contain at least one suite supported by local implementation; unknown IDs are ignored for negotiation but still transcript-bound	Reject with suiteNegotiationFailed
keyShares	Unique suiteId per entry, max 2 entries, each shareBytes must match its suiteId's expected length	Reject with invalidMessageFormat
selectedSuite	Must be in supportedSuites AND have matching keyShare	Reject with missingKeyShare
clientNonce/serverNonce	Must be 32 bytes	Reject with invalidMessageFormat
sigA/sigB	Must verify against respective identityPubKey	Reject with signatureVerificationFailed

TABLE S28
Encoding scope (V1 vs V2).

Component	Encoding	Length endianness	Used in v1 deployment
MessageA/MessageB wire bytes	V1 deterministic	little-endian	yes
Transcript hash input (v1)	V1 deterministic	little-endian	yes
Transcript hash input (v2)	V2 TLV canonical	big-endian	no (experimental)

TABLE S29
Suite identifier ranges and components.

Range	Category	Suite Components	Examples
0x00xx	Hybrid PQC (preferred)	X-Wing KEM, ML-DSA-65, AES-256-GCM, HKDF-SHA256	0x0001
0x01xx	Pure PQC	ML-KEM-768, ML-DSA-65, AES-256-GCM, HKDF-SHA256	0x0101
0x10xx	Classic	X25519 DHKEM, Ed25519, AES-256-GCM, HKDF-SHA256	0x1001
0xF0xx	Experimental	Reserved for testing	-

- `keyShares[]`: entries follow the suite preference order; only suites with provided shares appear
- All lists use 2-byte little-endian length prefix
- All integers use little-endian encoding
- Canonical encoding is byte-for-byte specified. Implementations **MUST NOT** reserialize with language-native encoders (e.g., JSON, PropertyList), as this may introduce non-determinism

Endianness rationale: V1 wire format uses little-endian to align with platform conventions (ARM64 native order). The V2 TLV format uses network byte order (big-endian) for TLV length fields to align with TLV conventions in IETF protocols. The two formats serve different purposes: V1 is the on-wire message encoding and is used as transcript input in v1 deployments (byte-for-byte deterministic). V2 TLV is an optional transcript-hashing format for forward compatibility and is exercised by regression tests; it is not used on the wire in v1.

Note: The 32-byte `clientNonce/serverNonce` in message fields are distinct from the 12-byte `aeadNonce` used internally by AES-GCM for authenticated encryption.

Suite Identifiers and Wire Format

A **suite** defines the handshake tuple (KEM, SIG, AEAD, KDF) used for the KEM-DEM envelope and transcript binding. Data-plane AEAD is negotiated separately and is fixed to AES-256-GCM in v1. Algorithm suite identifiers use a structured 16-bit wire format enabling forward compatibility:

Note: X-Wing is a hybrid KEM combining X25519 + ML-KEM-768; it does not include a signature algorithm. The suite identifier specifies the full primitive set, with ML-DSA-65 as the preferred signature algorithm when available. In v1, protocol signatures are bound to the suite group: ML-DSA-65 for PQC/hybrid suites and Ed25519 for classic suites. Secure Enclave P-256 ECDSA is used only for optional device proof-of-possession and is not used for the main protocol signatures (`sigA/sigB`).

**Post-rekey guarantee:* in Bootstrap-Assisted mode, business traffic remains blocked until PQC rekey succeeds. Runtime assurance labels reported by the implementation are `pqc_strict`, `bootstrap_assisted`, `legacy_classic`, and `unknown`.

Unknown suite identifiers are parsed as `unknown(wireId)` rather than causing parse failures. This allows older clients to gracefully reject unsupported suites during negotiation.

Transport/MTU considerations: handshake messages are length-prefixed and carried on reliable control channels (TCP or QUIC streams). Fragmentation and reassembly are handled by transport, not by the handshake format. QUIC datagrams are reserved for latency-critical media frames; for other payload types, senders must respect the maximum datagram size and chunk accordingly.

TABLE S30

Post-quantum coverage by policy and established suite (v1). “PQC confidentiality” refers to resistance against store-now-decrypt-later adversaries *while long-term KEM private keys remain uncompromised*; v1 does not claim PFS for KEM-based PQC suites.

Policy	Outcome	KEM / key establishment	Protocol signature (sigA/sigB)	PQ conf.	PQ auth.	PFS
default	PQC established	ML-KEM-768 (static recipient key)	ML-DSA-65	yes	yes	no
default	classic fallback	X25519 ephemeral DH	Ed25519	no	no	yes
strictPQC (Compliance)	PQC established	ML-KEM-768 (static recipient key)	ML-DSA-65	yes	yes	no
strictPQC (Compliance)	PQC unavailable	– (handshake fails)	–	–	–	–
strictPQC (Bootstrap-Assst.)	Control-only classic bootstrap for paired KEM-key recovery; mandatory PQC rekey	X25519 pre-rekey; ML-KEM-768 post-rekey	Ed25519 pre-rekey; ML-DSA-65 post-rekey	yes*	yes*	no

KEM-DEM Envelope

The sealed box structure encapsulates KEM-based authenticated encryption output with explicit DoS protection. Our construction follows the KEM-DEM (Key Encapsulation Mechanism + Data Encapsulation Mechanism) paradigm: `KEM.Encapsulate()` → HKDF-SHA256 → AES-256-GCM. We call this an “HPKE-inspired KEM-DEM envelope” to distinguish it from RFC 9180 HPKE, which includes additional features such as multiple modes (Base, PSK, Auth, AuthPSK), context exporters, and a more complex key schedule.

We model v1 as a constrained HPKE Base-mode instance: the KEM encapsulation yields a shared secret, HKDF-Extract/Expand derives an AEAD key/nonce with domain-separated info (role, suite ID, transcript hash), and AEAD provides confidentiality and ciphertext integrity under unique nonces. This makes the security dependencies explicit and isolates deviations from RFC 9180 to the simplified header/nonce/tag framing.

On Apple 26+ platforms, CryptoKit provides native HPKE with quantum-secure cipher suites including X-Wing (ML-KEM-768 + X25519). When available, implementations SHOULD use CryptoKit’s HPKE API directly rather than this compatibility envelope.

Security Goals:

- **PQC suites:** ML-KEM provides IND-CCA2 KEM security (per FIPS 203).
- **Classic suites:** X25519 ephemeral-DH encapsulation (DHKEM-style); security relies on the X25519/Gap-DH assumption and HKDF key separation.
- **Payload encryption:** INT-CTXT and IND-CPA via AES-256-GCM in v1 (compat KEM-DEM), and RFC 9180 HPKE AEAD in v2 (e.g., ChaCha20-Poly1305 in our classic provider).
- **Key separation:** HKDF with context-specific info parameters including role binding.

Not Covered (delegated to RFC 9180 HPKE on Apple 26+):

- Sender authentication modes (Auth, AuthPSK)
- Incremental AEAD for streaming

Used in our handshake (classic v2):

- HPKE exporter for deriving the per-session shared secret with explicit context binding

Format Versions:

The implementation supports two sealed box formats:

v1 (Compatibility KEM-DEM): Used when native HPKE is unavailable. Explicit nonce and tag fields.

Header (17 bytes):

magic	version	suite	flags
4B	1B	2B	2B
encLen	nonceLen	tagLen	ctLen
2B	1B	1B	4B

Body (v1):

encapsulatedKey	nonce	ciphertext	tag
encLen	12B	ctLen	16B

v2 (Native HPKE): Used with CryptoKit HPKE. Nonce and tag are embedded in the AEAD output.

Header (17 bytes):

magic	version	suite	flags
4B	1B	2B	2B
encLen	nonceLen	tagLen	ctLen
2B	0	0	4B

Body (v2):

encapsulatedKey	ciphertext (AEAD output)
encLen	ctLen (includes auth tag)

Version Detection: Parsers distinguish v1 from v2 by checking `nonceLen` and `tagLen`:

- v1: `nonceLen = 12, tagLen = 16`
- v2: `nonceLen = 0, tagLen = 0` (AEAD details encapsulated by library)

TABLE S31
Platform/runtime support matrix (Apple + cross-platform runtime policy).

Platform	PQC Provider	Algorithms	Notes
macOS 26+	ApplePQCProvider	ML-KEM-768, ML-KEM-1024 ¹ , ML-DSA-65, ML-DSA-87 ¹	Native CryptoKit
macOS 14–15	OQSPQCProvider	ML-KEM-768, ML-DSA-65	liboqs fallback
macOS 14–15	ClassicProvider	X25519, Ed25519	If liboqs unavailable
iOS 26+	ApplePQCProvider	ML-KEM-768, ML-KEM-1024 ¹ , ML-DSA-65, ML-DSA-87 ¹	Native CryptoKit
iOS 17–18	ClassicProvider	X25519, Ed25519	liboqs not bundled
Android 17+	Runtime tier policy	X-Wing ² → ML-KEM-768 (liboqs) → X25519	Prefer hybrid if available; fallback is eventuated + cooldown-limited
Android 14–16	Runtime tier policy	ML-KEM-768 (liboqs) → X25519	PQC first, then classic fallback under policy gate
Android 13 and below	Runtime tier policy	X25519, Ed25519	Classic-only baseline
Ubuntu 24–25	Runtime tier policy	X-Wing ² → ML-KEM-768 (liboqs) → X25519	Prefer hybrid if available; per-peer fallback cooldown enforced
Ubuntu 22–23	Runtime tier policy	ML-KEM-768 (liboqs) → X25519	PQC first, then classic fallback under policy gate
Ubuntu 20–21	Runtime tier policy	X25519, Ed25519	Classic-only baseline

TABLE S32
Cross-platform contract-consistency snapshot (ARTIFACT_DATE=2026-01-23). Generated by
Scripts/check_cross_platform_interop.py.

Check	Result	Notes
Suite catalog parity (IDs 0x0001, 0x0101, 0x0102, 0x1001, 0x1002)	PASS	iOS/macOS, Android, Ubuntu aligned
Wire endianness (UInt16 little-endian fields)	PASS	Encoding/decoding contract aligned
Signature-selection contract (PQC → ML-DSA-65, classic → Ed25519)	PASS	Cross-platform rule alignment
Timeout-triggered fallback blocked	PASS	Timeout not in default fallback gate
Per-peer fallback cooldown guard	PASS	Explicit cooldown paths present
Trust-pinning path	PASS	Pinned identity-fingerprint path present

TABLE S33
Cross-platform runtime suite-policy matrix used in this revision.

Platform range	Preferred suite order	Fallback policy note
Android 17+	X-Wing (if supported) → ML-KEM-768 (liboqs) → X25519	Classic fallback requires whitelist reason + per-peer cooldown
Android 14–16	ML-KEM-768 (liboqs) → X25519	PQC-first, classic as controlled fallback
Android 13 and below	X25519 only	Classic-only baseline
Ubuntu 24–25	X-Wing (if supported) → ML-KEM-768 (liboqs) → X25519	Classic fallback requires whitelist reason + per-peer cooldown
Ubuntu 22–23	ML-KEM-768 (liboqs) → X25519	PQC-first, classic as controlled fallback
Ubuntu 20–21	X25519 only	Classic-only baseline

Length limits enforce DoS protection:

- encLen ≤ 4096 bytes (sufficient for ML-KEM-768’s 1088-byte ciphertext)
- v1: nonceLen = 12 bytes, tagLen = 16 bytes (AES-GCM fixed)
- v2: nonceLen = 0, tagLen = 0 (embedded in ciphertext)
- ctLen ≤ 64KB (handshake phase, pre-authentication window)
- ctLen ≤ 256KB (post-authentication)

Parsing uses overflow-safe arithmetic and validates each field before allocation.

Platform Support Matrix

¹ ML-KEM-1024 and ML-DSA-87 are available in CryptoKit on Apple 26+ but not currently used by our implementation. ² X-Wing (wireId 0x0001) is negotiated only when runtime/provider support is available on both peers; otherwise tiered fallback selects ML-KEM-768 or classic according to policy.

Compatibility markers 0x0102 (ML-KEM-768-FS) and 0x1002 (P-256 marker) are parsed in the current Android/Ubuntu snapshot for forward compatibility and catalog parity, but remain non-negotiable by default until full end-to-end contract support is enabled.

Native PQC Integration

The ApplePQCCryptoProvider wraps CryptoKit ML-KEM and ML-DSA APIs, enabling platform-backed PQC suites and HPKE-based encapsulation. This construction (KEM → HKDF → AEAD) is inspired by HPKE but does not implement the full RFC 9180 specification. On Apple 26+ platforms, CryptoKit exposes HPKE cipher suites including X-Wing (ML-KEM-768 with X25519),

enabling a standards-aligned replacement for our compatibility envelope. This layer can then be replaced with direct CryptoKit PQ-HPKE calls.

Conditional Compilation Strategy

The `HAS_APPLE_PQC_SDK` flag gates **all** PQC type references. Importantly, `@available` only controls *runtime* availability; it does not prevent *compile-time* failures when the SDK lacks the PQC symbols.

For reproducible builds across toolchains, we intentionally **do not** enable `HAS_APPLE_PQC_SDK` by default in SwiftPM (because `.when(platforms: [.macOS])` does not reflect SDK availability and will break older Xcode builds). Instead, projects inject the flag from build settings **only** when compiling with the Apple 26 SDK (Xcode 26+): `OTHER_SWIFT_FLAGS = $(inherited) -DHAS_APPLE_PQC_SDK`

This keeps the codebase buildable on older Xcode versions (classic provider path), while enabling native CryptoKit PQC providers only when the correct SDK is present.

Security Event Emission

All cryptographic decisions emit structured events. The `SecurityEventEmitter` actor implements backpressure with per-subscriber queues and meta-event rate limiting to prevent recursive overflow.

Secure Enclave Integration

For hardware-backed signing, the `SigningCallback` protocol enables Secure Enclave integration. The `HandshakeDriver` prioritizes callback-based signing over raw key material, ensuring private keys never leave the Secure Enclave.

Availability and fallback. Secure Enclave-backed ML-DSA/ML-KEM keys are only available on macOS 26+ via CryptoKit. On macOS 14–15, PQC operations fall back to liboqs (software), and Secure Enclave is used only for P-256 ECDSA proof-of-possession keys. When Secure Enclave PQC is enabled, the implementation relies on CryptoKit key types `SecureEnclave.MLDSA*` and `SecureEnclave.MLKEM*`, both gated by macOS 26+ SDK and runtime availability checks.

Signing Key Hierarchy

The system supports two complementary signing mechanisms:

- 1) **CryptoProvider signing (Protocol Signature):**
Uses the active suite's signature algorithm (Ed25519 for classic, ML-DSA-65 for PQC) for protocol-level identity verification. Keys are managed by the `CryptoProvider` and stored in software. This is the primary signature used in `sigA/sigB` fields of handshake messages.
- 2) **Secure Enclave signing (Device PoP):**
Uses EC P-256 with ECDSA via `SecureEnclaveSigningCallback` to prove the peer controls a key stored in Secure Enclave. Private keys never leave the Secure Enclave hardware. Note: This provides proof-of-possession of a hardware-backed key, not full device attestation (Apple does not expose a general-purpose attestation API with certificate chains at the application layer). The security value derives from the key being pinned during initial pairing.

Implementation Note. `DeviceIdentityKeyManager` creates P-256 keys in Secure Enclave (when available) for hardware-backed device identity. These keys are used for the optional `seSigA/seSigB` proof-of-possession signatures, NOT for the primary protocol signatures (`sigA/sigB`). The primary protocol signatures use Ed25519 (classic) or ML-DSA-65 (PQC) keys generated by the `CryptoProvider`.

The `FallbackSigningCallback` provides automatic fallback from Secure Enclave to `CryptoProvider` when hardware signing is unavailable (e.g., on devices without Secure Enclave or when the key has not been provisioned).

Use Case Separation:

- `CryptoProvider` (Ed25519/ML-DSA): Primary protocol signatures, cross-platform interoperability, suite-negotiated
- Secure Enclave (P-256 ECDSA): Optional hardware-backed proof-of-possession, proving control of a non-exportable key

Both mechanisms can coexist in a single handshake: `CryptoProvider` for primary protocol signatures (`sigA/sigB`), Secure Enclave for optional hardware-backed proof-of-possession (`seSigA/seSigB`).

Signature Verification Rules:

- Primary signatures (`sigA/sigB`) are mandatory and must verify against the peer's `identityPubKey`.
- For paired peers, `identityPubKey` must match the pinned value from initial pairing.
- Optional SE signatures (`seSigA/seSigB`) elevate trust when present and valid.
- Verification order: primary signature, identity pinning, optional SE signature.

TABLE S34
Default-policy classic fallback eligibility (paired peers).

Reason code / class	Eligible?	Gate semantics and notes
<code>pqcProviderUnavailable</code>	ALLOW	Local provider/API instantiation failure in the <i>pre-send</i> branch; emits <code>cryptoDowngrade</code> with reason and cooldown context.
<code>suiteNotSupported</code>	ALLOW	Local capability mismatch in the <i>pre-send</i> branch (e.g., no supported PQC suite for the peer's pinned KEM material); emits <code>cryptoDowngrade</code> .
<code>suiteNegotiationFailed</code>	ALLOW (pre-send only)	Only the local <i>pre-send</i> construction/selection branch is fallback-eligible; if observed after remote input it is treated as network-derived and fallback is denied.
<code>timeout</code>	DENY	Explicitly blocked under default policy to prevent loss-induced downgrade.
<code>signatureVerificationFailed</code>	DENY	After-send/network-derived; emit typed failure events only (not downgrade-labeled).
<code>identityMismatch</code>	DENY	After-send/network-derived; emit typed failure events only (not downgrade-labeled).
<code>replayDetected</code>	DENY	After-send/network-derived; emit typed failure events only (not downgrade-labeled).

TABLE S35
ML-DSA-65 Key Sizes (FIPS 204 Compliance).

Component	Expected	Measured	Status
Public Key	1952 B	1952 B	OK
Secret Key	4032 B	4032 B	OK
Signature	3309 B	3309 B	OK

Pre-Negotiation Signature Selection and Two-Attempt Strategy

A fundamental challenge in our protocol is the “chicken-and-egg” problem: `sigA` must be generated *before* suite negotiation completes, yet the signature algorithm should be consistent with the negotiated suite. We resolve this through pre-negotiation signature selection and a two-attempt strategy.

Pre-Negotiation Signature Selection: The signature algorithm for `sigA` is determined by the `offeredSuites` in `MessageA`, not by the final `selectedSuite`. The `PreNegotiationSignatureSelector` builds offered suites based on the attempt strategy and selects the appropriate signature algorithm (ML-DSA-65 for PQC suites, Ed25519 for classic).

Homogeneity Invariant: Each attempt's `offeredSuites` must be homogeneous with respect to `sigAAlgorithm`: ML-DSA-65 requires all PQC suites; Ed25519 requires all classic suites. This invariant is enforced at compile-time through the type system and at runtime through `HandshakeDriver` initialization validation.

Two-Attempt Strategy: To support interoperability between PQC-capable and classic-only devices, we employ a two-attempt strategy: (1) PQC attempt with ML-DSA-65 signatures, then (2) classic fallback with Ed25519 if the PQC attempt fails due to provider unavailability or *local pre-send* suite-negotiation failure. In particular, for a classic-only peer without a pinned PQC KEM public key from pairing, the PQC attempt fails locally during `MessageA` construction (no key shares → `suiteNegotiationFailed`), so fallback is immediate and is not driven by network timeout.

Strict Policy Modes:

- **Compliance Mode:** strict fail-closed semantics; no classic establishment.
- **Bootstrap-Assisted Mode:** for paired peers with missing or stale KEM identity keys, allow one temporary classic control channel (`pairingIdentityExchange` only), then require successful PQC rekey before business traffic.

Fallback Security (default policy classic fallback): Not all failures trigger default-mode classic fallback.

Local-only definition: In this paper, “local-only” means the failure is observed at the initiator-side attempt-switch boundary before emitting `MessageA` in the current attempt (i.e., before any on-wire handshake bytes are sent under that attempt). This prevents an active network attacker from converting packet loss or malformed inputs into a fallback-eligible condition.

DoS note: Endpoint load or resource exhaustion can still cause local provider instantiation to fail. We treat such cases as availability (DoS) rather than a secrecy failure; when fallback is allowed it is always explicit (downgrade-labeled), rate-limited, and therefore audit-visible. Deployments that must never establish classic business sessions should use strict Compliance Mode.

Timeout-based fallback is explicitly blocked to prevent attackers from forcing downgrade through packet dropping. Per-peer fallback is rate-limited (default 5-minute cooldown) to prevent rapid downgrade cycling.

Strict Bootstrap-Assisted triggers:

- `suiteNegotiationFailed` with paired trust and missing KEM identity key.
- `timeout` with paired trust and existing KEM key record (stale-key recovery).

During this strict recovery path, non-control business messages are blocked until PQC rekey succeeds; recovery completion/failure is emitted as `handshake_fallback` security events.

ML-DSA Key Sizes

Key Size Reference

¹ Apple CryptoKit uses a seed-based compact format for private key serialization (`integrityCheckedRepresentation`). This is more storage-efficient than the FIPS 203/204 expanded format. Public keys use `rawRepresentation` which matches FIPS standard sizes. Measurements performed on macOS 26.0 (Tahoe) SDK.

TABLE S36
Key size reference.

Algorithm	Public Key	Private Key (Apple) ¹	Private Key (FIPS)	Ciphertext/Signature
ML-KEM-768	1184 B	96 B	2400 B	1088 B
ML-KEM-1024	1568 B	128 B ²	3168 B	1568 B
ML-DSA-65	1952 B	64 B	4032 B	3309 B
ML-DSA-87	2592 B	96 B ²	4896 B	4627 B
X25519	32 B	32 B	-	32 B
Ed25519	32 B	32 B	-	64 B

TABLE S37
Security event types.

Event Type	Severity	Trigger
cryptoProviderSelected	info/warning	Provider factory selection
cryptoDowngrade	warning	PQC to classic fallback
handshakeFailed	warning	Any handshake failure
signatureVerificationFailed	high	Invalid peer protoSignature
secureEnclaveVerificationFailed	warning	Invalid secureEnclaveSignature
identityMismatch	high	identityPubKey does not match pinned key
contextZeroized	info	Sensitive material cleared
suiteNegotiationFailed	warning	No common suite found
unexpectedStateTransition	high	Actor reentrancy detected

TABLE S38
Boundary-stress cases: model scope, expected audit signals, and operator actions.

Boundary-stress case	Model status	Expected signal surface	Required operator action	Claim impact
Pairing social engineering (user pins attacker key)	Out-of-model (pairing assumption failure)	No cryptographic alert at pairing time; later key-change conflicts may appear as <code>identityMismatch</code>	Revoke trust record, require authenticated re-pairing (QR/PIN or equivalent), quarantine previously trusted channel	Authentication claims do not apply until re-pair completes
Endpoint compromise (read-only key/material exposure)	Partially out-of-model (endpoint integrity degraded)	Potential abnormal <code>cryptoDowngrade/handshakeFailed</code> bursts; possible forensics-only detection	Enter strict mode where feasible, rotate affected long-term keys, investigate endpoint posture before restoring trust	Secrecy claims degrade to compromise window assumptions
Endpoint compromise (signing-capable attacker)	Out-of-model for protocol-level peer trust	Handshake-level signatures may still verify; compromise usually inferred from external telemetry	Immediate endpoint quarantine, key revocation/rotation, forced re-pairing for all peers	Mutual-authentication guarantees no longer hold for compromised endpoint
Audit-log injection/truncation attempt on local store	In-model for integrity checking of emitted records; out-of-model for fully privileged host tamper	Hash-chain discontinuity or missing mandatory fields for downgrade events	Treat chain break as incident, preserve evidence snapshot, fail closed for automated compliance checks	Audit replayability claims hold only while logging substrate integrity assumptions hold

² Not measured directly; projected from the seed-based pattern observed in ML-KEM-768/ML-DSA-65 (ratio of FIPS expanded size to Apple compact size). These algorithms are available in CryptoKit but not exercised by our current benchmark suite.

Security Event Types

Boundary Stress Cases and Operator Runbook

Rows marked out-of-model are not covered by theorem-level security claims in main-paper Section 5; they are included to make deployment response obligations explicit.

X-Wing Wire Size (Measured)

X-Wing is a hybrid KEM combining X25519 + ML-KEM-768. In our v1 handshake, the initiator carries only a KEM ciphertext in MessageA; thus X-Wing increases the MessageA keyshare from 1088 B (ML-KEM-768) to 1216 B in the 2026-01-23 snapshot, while MessageB remains keyshare-free. We measure X-Wing directly via the native CryptoKit provider (`wireId 0x0001`) and report per-field sizes in Supplementary Table S14 (`MessageA.XWing/MessageB.XWing`), using the X-Wing snapshot dated 2026-01-23. The field breakdown is:

- MessageA.XWing: 3309 (signature) + 1216 (keyshare) + 1956 (identity) + 160 (overhead) = 6641 B
- MessageB.XWing: 3309 (signature) + 0 (keyshare) + 1956 (identity) + 154 (overhead) = 5419 B
- Finished: 38 B

Total handshake size is **12,136 B**. Relative to the payload-only liboqs baseline (12,002 B), the +134 B delta reflects +128 B X-Wing keyshare expansion and +6 B framing overhead in this snapshot. The CryptoKit PQC payload-only baseline is 12,009 B in this snapshot (Supplementary Table S14).

Reproducibility

Test Commands.

- One-shot evaluation: `Scripts/run_paper_eval.sh`
- Individual suite run: `swift test` with `-filter TestName`
- Representative suite names:
 - `HandshakeBenchmarkTests`
 - `HandshakeFaultInjectionBenchTests`
 - `PolicyDowngradeBenchTests`
 - `MessageSizeSnapshotTests`
- CSV outputs are date-stamped under `Artifacts/`. To pin one dataset, set `ARTIFACT_DATE=2026-01-23` or `SKYBRIDGE_ARTIFACT_DATE=2026-01-23`.

System-impact artifact hygiene (snapshot 2026-01-23). For `Artifacts/system_impact_2026-01-23.csv`, we removed one duplicated embedded header row (a legacy append artifact) before table generation; no other data rows were modified. Final file summary:

- 13,801 total lines (1 header + 13,800 data rows)
- Per-suite row counts consistent with benchmark design: `ideal=1000`, `rtt100_j50=1000`, `rtt50_j20=2600`
- No fully duplicated data rows were found

Repeatability (Multi-batch). Repeatability tables report observed batch count B and (when $B \geq 2$) mean $\pm$ 95% confidence intervals across independent batches. To reproduce multi-batch CIs, rerun with process restarts: `ARTIFACT_DATE=2026-01-23 SKYBRIDGE_BENCH_BATCHES=3-5 bash Scripts/run_paper_eval.sh`.

Real-network micro-study (External validity). We use two lightweight scripts:

- `Scripts/run_real_network_probe.swift`: STUN RTT + conservative NAT classification
- `Scripts/run_real_network_e2e.swift`: TCP connect + first-byte + completion timing for payload-only handshake baselines (classic 687 B; PQC 12,002 B liboqs / 12,009 B CryptoKit; transport uses a fixed 4-byte length prefix)

Pin filenames with `ARTIFACT_DATE=2026-01-23`, run across labels (same Wi-Fi / cross-NAT / hotspot), then aggregate with `python3 Scripts/aggregate_realnet.py`.

iOS on-device microbench import. Run the iOS app path `Settings → PQC → Self-test/Bench` (warmup=10, N=1000, B=3). Export:

- `ios_microbench_<date>_ipad16_3.json`
- `ios_microbench_<date>_iphone17_1.json`

Use `<date>=2026-01-23` for the frozen artifact snapshot. Place files under `Artifacts/`, then run `python3 Scripts/aggregate_ios_microbench.py`.

Kernel-level emulation cross-check. Run `Scripts/run_network_emulation_kernel.sh` (requires `sudo/network-admin` capability), then aggregate with `python3 Scripts/aggregate_kernel_emulation.py`. Shared profile IDs come from `Scripts/netem_profiles.json`; reorder is netem-only and dummynet entries are marked *n/a*.

Note on inbound reachability. Some home networks do not provide an inbound-reachable IPv4 address (e.g., double NAT behind an ISP router, CGNAT, DS-Lite, or WAN IPv4 shown as 0.0.0.0). In these cases, IPv4 port-forwarding experiments will fail (client samples report `timeout` with empty `connect_ms`). For a true cross-NAT condition without modifying upstream equipment, prefer IPv6 direct connectivity (when available) with an explicit IPv6 firewall allow-rule for TCP 44444. If only an overlay/relay path is feasible, label the condition accordingly (e.g., `cross_nat_via_relay`) and interpret results as overlay-mediated connectivity rather than raw Internet inbound reachability.

Regression Testing

Transcript integrity uses V1 (deterministic) and V2 (TLV canonical) encoding formats, validated with 14 test cases (100% pass rate). TLV encoding uses 1-byte tags, 4-byte big-endian length fields, and variable-length values. The complete regression matrix validates Requirements 12.1–12.6; all 15 regression tests pass, and all 3 compile-fail harness tests correctly produce compile errors.

Key Properties: (1) P-256 cannot be used as a protocol signature algorithm (compile-time enforced); (2) PQC attempts use only PQC suites, classic attempts use only classic suites; (3) Default-mode classic fallback is limited to whitelisted local capability errors, while strict Bootstrap-Assisted recovery is limited to paired KEM-key refresh triggers; (4) Legacy P-256 requires authenticated channel or existing TrustRecord; (5) All fallback and legacy acceptance events include full audit context.

TABLE S39
Formal-to-code traceability for theorem-facing migration guarantees.

Formal claim / lemma	Model events	Implementation anchor	Artifact/check anchor
'TimeoutCannotTriggerClassicFallback'	TimeoutObserved + FallbackDenied(timeout)	Sources/SkyBridgeCore/P2P/ TwoAttemptHandshakeManager.swift (timeout deny path and fallback blacklist)	Artifacts/ migration_threat_harness_ 2026-01-23.csv (timeout attack rows); Fig. 5
'FallbackRateLimitEnforced'	CooldownChecked + CooldownPermit	Sources/SkyBridgeCore/P2P/ TwoAttemptHandshakeManager.swift (per-peer cooldown gate)	Artifacts/ migration_fleet_behavior_ 2026-01-23.csv; Table S5
'DowngradeEventRequiredWithContext'	DowngradeEventCtx	Sources/SkyBridgeCore/Security/ SecurityEventEmitter.swift + Sources/SkyBridgeCore/P2P/ TwoAttemptHandshakeManager.swift (downgrade event fields)	Artifacts/ migration_threat_harness_ 2026-01-23.csv (ECR/PCR); Table S9
'BootstrapControlOnlyUntilPQCRekey'	BootstrapControlChannel, BusinessTrafficBlocked, PQCRekeyCompleted, BusinessTrafficEnabled	Sources/SkyBridgeCore/P2P/ TwoAttemptHandshakeManager.swift + Sources/SkyBridgeCore/P2P/ HandshakeDriver.swift (strict bootstrap/rekey gating)	formal/ skybridge_v2.spthy; Artifacts/ tamarin_skybridge_v2_ summary_2026-01-23.txt

Formal-to-Code Traceability

Extended Future Directions

Moved from main paper Sec. 8.2 due to page limit.

- 4) **Traffic Analysis Mitigations:** Reduce two-attempt identifiability via optional padding or constant-size framing, while preserving strict Compliance-Mode semantics for deployments that require zero classic establishment.
- 5) **Real-Network and Kernel-Emulation Expansion:** Extend the current micro-study and kernel cross-check to larger cohorts, more ISP topologies, and longer sessions; compare TCP vs QUIC control channels under matched profile IDs and evaluate stability over multi-day runs. Immediate targets are to add at least one additional non-overlay real-network labels and to calibrate dummynet profile separation with non-localhost endpoints plus independent packet-capture validation.
- 6) **Formal Verification:** Apply model checking to the handshake state machine to prove absence of deadlocks and race conditions.
- 7) **Performance Optimization:** Profile ML-KEM-768 encapsulation latency and explore hardware acceleration opportunities.
- 8) **Certificate-Based Identity:** Integrate with device certificates for enterprise deployment scenarios.

Artifact Map

Table S40 maps implementation components to source files in the artifact repository (<https://github.com/billlza/Skybridge-Compass>, ref=251a162eb97d, commit=251a162eb97d).

TABLE S40
Implementation artifact map.

Component	File (Lines)
Provider Protocol	Sources/SkyBridgeCore/P2P/CryptoProviderProtocol.swift (L24–113)
Handshake Types	Sources/SkyBridgeCore/P2P/HandshakeTypes.swift (L297–320)
Provider Factory	Sources/SkyBridgeCore/P2P/CryptoProviderFactory.swift (L18–179)
Handshake Driver	Sources/SkyBridgeCore/P2P/HandshakeDriver.swift (L258–338, L1042–1145)
Handshake Context	Sources/SkyBridgeCore/P2P/HandshakeContext.swift (L27–100, L862–933)
Secure Bytes	Sources/SkyBridgeCore/P2P/SecureBytes.swift (L25–99)
Message Signatures	Sources/SkyBridgeCore/P2P/HandshakeMessages.swift (L557–569, L801–821)
Apple PQC Provider	Sources/SkyBridgeCore/P2P/Providers/ApplePQCProvider.swift (L27–152)
Security Events	Sources/SkyBridgeCore/Security/SecurityEventEmitter.swift (L22–136)
SE Signing	Sources/SkyBridgeCore/P2P/SecureEnclaveSigningCallback.swift (L40–94)
Signature Selector	Sources/SkyBridgeCore/P2P/PreNegotiationSignatureSelector.swift (L69–96)
Route-Aware Discovery	Sources/SkyBridgeCore/P2P/P2PDiscoveryService.swift (L278–457)
Connection State Guarding	Sources/SkyBridgeCore/DeviceDiscovery/UnifiedOnlineDeviceManager.swift (L940–989)
Active Transfer Route Selection	Sources/SkyBridgeCore/FileTransfer/FileTransferManager.swift (L104–213)